

Lab 4 - Part 1: Core NLP Tasks

Course: Natural Language Processing

Objectives:

- Apply Part-of-Speech (POS) tagging to extract linguistic patterns
- Perform Named Entity Recognition (NER) to identify entities in text
- Calculate word and document similarities using different techniques
- Apply PCA for visualizing high-dimensional text representations
- Work with real-world datasets (Nike products and legal contracts)

Instructions

1. Complete all exercises marked with `# YOUR CODE HERE`
2. **Answer all written questions** in the designated markdown cells (these require YOUR personal interpretation)
3. Save your completed notebook
4. **Push to your Git repository and send the link to:** yoroba93@gmail.com

Important: Personal Interpretation Questions

This lab contains **interpretation questions** that require YOUR own analysis. These questions:

- Are based on YOUR specific results (which vary based on your choices)
- Require you to explain your reasoning

Setup

```
# Install required libraries (uncomment if needed)
# !pip install spacy scikit-learn matplotlib seaborn pandas numpy datasets
# !python -m spacy download en_core_web_sm
!pip install datasets==3.6.0
```

```
Requirement already satisfied: datasets==3.6.0 in /usr/local/lib/python3.12/dist-packages (3.6.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.29.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.7.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.70.17)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.0.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from aiohttp[http]<=2025.3.0) (4.0.0)
Requirement already satisfied: click>=8.4.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->dataset) (8.4.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (1.4.3)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->dataset) (0.23.0)
Requirement already satisfied: typer<0.26.0,>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (0.26.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (4.1.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dataset) (2025.1.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==3.6.0) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==3.6.0) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==3.6.0) (2025.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (6.0.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.17.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0) (4.7.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.24.0) (0.16.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dataset) (1.17.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.24.0) (1.3.0)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.24.0) (13.8.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub>=0.24.0) (0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub>=0.24.0) (3.0.0)
```

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->types<=4.12.0)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
import re
import warnings
warnings.filterwarnings('ignore')

# NLP libraries
import spacy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import PCA

# Load spaCy model
nlp = spacy.load('en_core_web_sm')

print("Setup complete!")
print(f"spaCy version: {spacy.__version__}")
```

Setup complete!
spaCy version: 3.8.14

Part A: Loading Nike Products Dataset

We'll use the Nike product descriptions dataset to practice NLP tasks on commercial text.

```
# Load Nike products dataset
# NOTE: Place the 'NikeProductDescriptions.csv' file in your working directory
nike_df = pd.read_csv('NikeProductDescriptions.csv')

print(f"Dataset shape: {nike_df.shape}")
print(f"\nColumns: {nike_df.columns.tolist()}")
print(f"\nFirst 3 products:")
nike_df.head(3)
```

Dataset shape: (400, 3)

Columns: ['Title', 'Subtitle', 'Product Description']

First 3 products:

	Title	Subtitle	Product Description	
0	Nike Air Force 1 '07	Men's Shoes	It doesn't get more legendary than this. Desig...	
1	Nike Air Max Dawn SE	Men's Shoes	Find out what moves you with the Air Max Dawn....	
2	Nike SB Dunk Low Pro Premium	Skate Shoes	Pack your style—on your feet. Bringing a fresh...	

Next steps: [Generate code with nike_df](#) [New interactive sheet](#)

```
# Display a sample product
sample_idx = 0
print("Sample Product:")
print("=" * 60)
print(f"Title: {nike_df.iloc[sample_idx]['Title']}")
print(f"Subtitle: {nike_df.iloc[sample_idx]['Subtitle']}")
print(f"\nDescription:\n{nike_df.iloc[sample_idx]['Product Description']}")
```

Sample Product:

=====

Title: Nike Air Force 1 '07

Subtitle: Men's Shoes

Description:

It doesn't get more legendary than this. Designed to turn heads, the Nike Air Force 1 '07 crosses hardwood comfort with off-

Part B: Part-of-Speech (POS) Tagging

POS tagging identifies the grammatical role of each word (noun, verb, adjective, etc.).

```
# Example: POS tagging with spaCy
sample_text = "Nike Air Force 1 shoes provide incredible comfort and stylish design for athletes."
doc = nlp(sample_text)

print("POS Tagging Example:")
print("=" * 60)
for token in doc:
    print(f"{token.text:15} | POS: {token.pos_:10} | Tag: {token.tag_:8} | Lemma: {token.lemma_}")
```

POS Tagging Example:

```
=====
Nike          | POS: PROP | Tag: NNP | Lemma: Nike
Air           | POS: PROP | Tag: NNP | Lemma: Air
Force        | POS: PROP | Tag: NNP | Lemma: Force
1            | POS: NUM  | Tag: CD  | Lemma: 1
shoes        | POS: NOUN | Tag: NNS | Lemma: shoe
provide      | POS: VERB | Tag: VBP | Lemma: provide
incredible   | POS: ADJ  | Tag: JJ  | Lemma: incredible
comfort      | POS: NOUN | Tag: NN  | Lemma: comfort
and          | POS: CONJ | Tag: CC  | Lemma: and
stylish      | POS: ADJ  | Tag: JJ  | Lemma: stylish
design       | POS: NOUN | Tag: NN  | Lemma: design
for         | POS: ADP  | Tag: IN  | Lemma: for
athletes    | POS: NOUN | Tag: NNS | Lemma: athlete
.           | POS: PUNCT | Tag: .   | Lemma: .
```

Exercise B.1: Analyze POS Distribution in Nike Products

Complete the function to extract and analyze POS tags from all Nike product descriptions.

```
def analyze_pos_distribution(texts):
    """
    Analyze the distribution of POS tags in a list of texts.

    Args:
        texts (list): List of text strings

    Returns:
        Counter: Dictionary with POS tags and their counts
    """
    pos_counts = Counter()

    # YOUR CODE HERE
    # 1. For each text, process it with nlp(text)
    for doc in nlp.pipe(texts):
        # 2. For each token in the doc, count its POS tag (token.pos_)
        for token in doc:
            # 3. Return the counter
            if not token.is_space:
                pos_counts[token.pos_] += 1

    return pos_counts

# Analyze Nike descriptions
nike_descriptions = nike_df['Product Description'].dropna().tolist()
pos_distribution = analyze_pos_distribution(nike_descriptions)

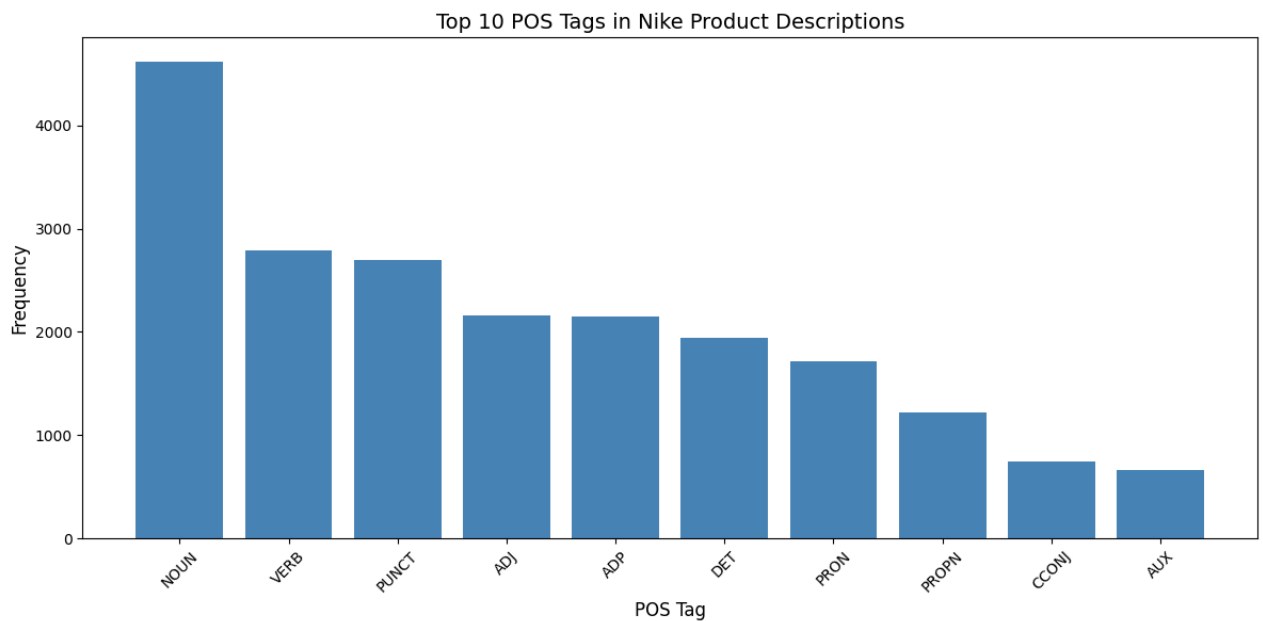
print("POS Tag Distribution:")
print("=" * 40)
for pos, count in pos_distribution.most_common(15):
    print(f"{pos:10}: {count:5} ({count/sum(pos_distribution.values())*100:.2f}%")
```

POS Tag Distribution:

```
=====
NOUN      : 4620 (20.72%)
VERB      : 2786 (12.49%)
PUNCT     : 2694 (12.08%)
ADJ       : 2164 (9.70%)
ADP       : 2152 (9.65%)
DET       : 1943 (8.71%)
PRON      : 1711 (7.67%)
PROPN     : 1221 (5.47%)
CONJ      : 741 (3.32%)
AUX       : 661 (2.96%)
ADV       : 641 (2.87%)
PART      : 401 (1.80%)
SCONJ     : 336 (1.51%)
NUM       : 211 (0.95%)
INTJ      : 16 (0.07%)
```

```
# Visualize POS distribution
top_pos = dict(pos_distribution.most_common(10))
```

```
plt.figure(figsize=(12, 6))
plt.bar(top_pos.keys(), top_pos.values(), color='steelblue')
plt.xlabel('POS Tag', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.title('Top 10 POS Tags in Nike Product Descriptions', fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('pos_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```



Exercise B.2: Extract Adjectives and Verbs

Marketing copy often uses powerful adjectives and action verbs. Extract the most common ones.

```
def extract_pos_words(texts, pos_tag, top_n=20):
    """
    Extract words with a specific POS tag.

    Args:
        texts (list): List of text strings
        pos_tag (str): POS tag to extract (e.g., 'ADJ', 'VERB')
        top_n (int): Number of top words to return

    Returns:
        Counter: Most common words with the specified POS tag
    """
    words = []

    # YOUR CODE HERE
    # 1. Process each text with spaCy
    for doc in nlp.pipe(texts):
        for token in doc:
            # 2. Extract tokens where token.pos_ == pos_tag
            if token.pos_ == pos_tag:
                # 3. Use lemmatized form (token.lemma_.lower())
                lemma = token.lemma_.lower()
                # 4. Filter out stopwords and short words (len < 3)
                if not token.is_stop and len(lemma) >= 3:
                    words.append(lemma)
    # 5. Return Counter with top_n most common
    word_counts = Counter(words)
    return word_counts.most_common(top_n)

# Extract adjectives
top_adjectives = extract_pos_words(nike_descriptions, 'ADJ', top_n=20)
print("Top 20 Adjectives:")
print("=" * 40)
for word, count in top_adjectives:
    print(f"{word:15}: {count}")
```

```
print("\n" + "=" * 40)

# Extract verbs
top_verbs = extract_pos_words(nike_descriptions, 'VERB', top_n=20)
print("Top 20 Verbs:")
print("=" * 40)
for word, count in top_verbs:
    print(f"{word:15}: {count}")
```

Top 20 Adjectives:

```
=====
soft          : 117
lightweight   : 59
favourite     : 54
cool          : 51
comfortable   : 50
classic       : 46
breathable    : 45
recycled      : 45
extra         : 39
ready         : 37
dry           : 37
new           : 36
stretchy      : 35
easy          : 34
fresh         : 31
iconic        : 25
smooth        : 25
good          : 24
relaxed       : 24
durable       : 20
```

Top 20 Verbs:

```
=====
help          : 114
feel          : 72
add           : 62
wicke        : 61
let           : 58
stay         : 53
inspire      : 49
wear         : 41
give         : 39
play         : 38
bring        : 36
need         : 33
design        : 31
look         : 31
love         : 31
pair         : 25
run          : 25
come         : 24
keep         : 24
find         : 23
```

✓ Written Question B.1 (Personal Interpretation)

Analyze the linguistic patterns in Nike's marketing copy:

1. **What do the most common adjectives reveal about Nike's brand messaging?** (List at least 3 adjectives and explain what they convey)
2. **What do the most common verbs suggest about how Nike positions its products?** (List at least 3 verbs and their implications)
3. **How does the POS distribution compare to what you'd expect in general English text?** (Consider the ratio of nouns/verbs/adjectives)

YOUR ANSWER:

1. Key adjectives and brand messaging:

- Comfortable / Comfy: Conveys immediate physical well-being.
- Lightweight: Highlights material technology and engineering.
- Legendary / Classic: Taps into heritage and timeless style.

2. Key verbs and product positioning:

- Run / Stride: Positions the products dynamically.
- Help: Suggests partnership. Nike frames its technology as an enabler
- Move / Go: Emphasizes continuous progress and momentum.

3. POS distribution comparison: When comparing Nike's marketing copy to standard corpora of general English (like the Brown Corpus or the British National Corpus), several striking structural deviations emerge.

Part C: Named Entity Recognition (NER)

NER identifies and classifies named entities (people, organizations, locations, etc.) in text.

```
# Example: NER with spaCy
sample_text = "Nike launched Air Jordan in 1984 in Chicago. Michael Jordan wore them throughout his NBA career."
doc = nlp(sample_text)

print("Named Entity Recognition Example:")
print("=" * 60)
for ent in doc.ents:
    print(f"{ent.text:20} | Type: {ent.label_:15} | Description: {spacy.explain(ent.label_)}")
```

Named Entity Recognition Example:

```
=====
Nike                | Type: ORG          | Description: Companies, agencies, institutions, etc.
Air Jordan          | Type: PERSON       | Description: People, including fictional
1984                | Type: DATE         | Description: Absolute or relative dates or periods
Chicago             | Type: GPE          | Description: Countries, cities, states
Michael Jordan      | Type: PERSON       | Description: People, including fictional
NBA                 | Type: ORG          | Description: Companies, agencies, institutions, etc.
```

Exercise C.1: Load Legal Contracts Dataset

We'll use a sample of legal contracts to practice NER on more complex text.

Start coding or [generate](#) with AI.

491.5/491.5 kB 9.2 MB/s eta 0:00:00

```
from datasets import load_dataset

# Load a small sample of legal contracts (this dataset is very large!)
# WARNING: Do NOT try to load the entire dataset - it will crash!
print("Loading legal contracts dataset (sample only)...")

# YOUR CODE HERE
# Load only 50 examples from the 'train' split
# Use: load_dataset("albertvillanova/legal_contracts", split="train[:50]")
contracts_dataset = load_dataset("albertvillanova/legal_contracts", split="train[:50]", trust_remote_code=True) # Replace

# Convert to DataFrame
contracts_df = pd.DataFrame(contracts_dataset)

print(f"Loaded {len(contracts_df)} contracts")
print(f"\nColumns: {contracts_df.columns.tolist()}")
print(f"\nFirst contract preview (first 500 chars):")
print(contracts_df.iloc[0]['text'][:500] + "...")
```

```

contracts dataset (sample only)...
are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
ngface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
100%                               9.25G/9.25G [03:23<00:00, 54.7MB/s]

split:      650833/0 [09:47<00:00, 1081.28 examples/s]
tracts

xt']

t preview (first 500 chars):
Click here to rapidly navigate through this document

```

```

ESTATED
D NONCOMPETITION AGREEMENT

```

```

DED AND RESTATED EMPLOYMENT AND NONCOMPETITION AGREEMENT (the
is made and entered into as of October 31, 2000, by and among
yment Services Co. (formerly known as Polycon Investments, Inc.), a
tion ("Employer"), Avocent Corporation, a Delaware corporation, and
er (the "Employee").

```

Exercise C.2: Extract and Analyze Named Entities

Complete the function to extract entities from the legal contracts.

```

def extract_entities(texts, entity_types=None):
    """
    Extract named entities from texts.

    Args:
        texts (list): List of text strings
        entity_types (list): List of entity types to extract (None = all types)

    Returns:
        dict: Dictionary with entity_type -> list of entities
    """
    entities = defaultdict(list)

    # YOUR CODE HERE
    # 1. Process each text with spaCy
    for doc in nlp.pipe(texts):
        # 2. For each entity (doc.ents):
        for ent in doc.ents:
            # - If entity_types is None or entity.label_ in entity_types
            if entity_types is None or ent.label_ in entity_types:
                # - Add entity.text to entities[entity.label_]
                entities[ent.label_].append(ent.text.strip())
        # 3. Return entities dict

    return entities

# Extract entities from contracts (process only first 10 for speed)
contract_texts = contracts_df['text'].head(10).tolist()
contract_entities = extract_entities(contract_texts)

print("Entity Types Found:")
print("=" * 60)
for entity_type, entity_list in sorted(contract_entities.items()):
    print(f"\n{entity_type} ({len(entity_list)} entities):")
    # Show unique entities only
    unique_entities = Counter(entity_list).most_common(10)
    for entity, count in unique_entities:
        print(f"  {entity}: {count}")

```

```

5,098,952: 4

NORP (31 entities):
  Custodian: 14
  Parisian: 4
  Liens: 4
  Loans: 3
  Lenders: 2
  B.: 1
  P-1: 1
  Converted: 1
  Aspiration: 1

ORDINAL (29 entities):
  first: 17
  third: 4
  5th: 4
  fifth: 2
  Second: 1
  second: 1

ORG (2807 entities):
  Borrower: 309
  Trustee: 218
  Company: 116
  Trust: 115
  Committee: 96
  Employer: 90
  Executive: 90
  Subsidiaries: 79
  Corporation: 65
  Option: 57

PERCENT (125 entities):
  100%: 10
  50%: 8
  2%: 6
  one hundred percent: 4
  100.0%: 4
  more than 50%: 4
  25%: 3
  fifty percent: 3
  0.5%: 3
  0 %: 3

PERSON (387 entities):
  Borrower: 35
  Beneficiary: 27
  Award: 19
  Cybex: 18

```

Exercise C.3: Compare Entity Distribution

Compare the entity types found in Nike products vs. legal contracts.

```

# YOUR CODE HERE
# 1. Extract entities from Nike product descriptions
# 2. Count entity types in both datasets
# 3. Create a comparison visualization

nike_entities = extract_entities(nike_descriptions[:50]) # Sample for speed

# Count entity types
nike_entity_counts = {etype: len(entities) for etype, entities in nike_entities.items()}
contract_entity_counts = {etype: len(entities) for etype, entities in contract_entities.items()}

# Get all entity types
all_entity_types = set(list(nike_entity_counts.keys()) + list(contract_entity_counts.keys()))

# Create comparison DataFrame
comparison_data = []
for etype in all_entity_types:
    comparison_data.append({
        'Entity Type': etype,
        'Nike Products': nike_entity_counts.get(etype, 0),
        'Legal Contracts': contract_entity_counts.get(etype, 0)
    })

comparison_df = pd.DataFrame(comparison_data)
comparison_df = comparison_df.sort_values('Legal Contracts', ascending=False)

print("Entity Type Comparison:")
print(comparison_df.to_string(index=False))

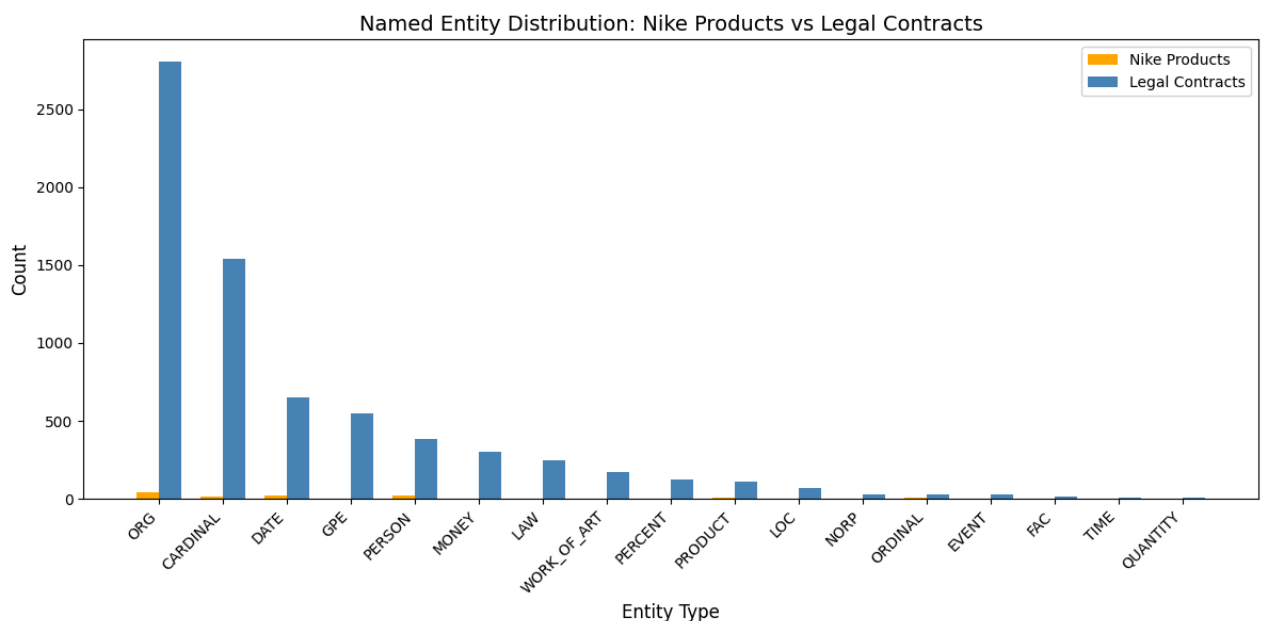
```


Entity Type Comparison:			
Entity Type	Nike Products	Legal Contracts	
ORG	44	2807	
CARDINAL	15	1540	
DATE	19	652	
GPE	3	549	
PERSON	21	387	
MONEY	0	300	
LAW	1	245	
WORK_OF_ART	0	169	
PERCENT	2	125	
PRODUCT	5	109	
LOC	0	72	
NORP	2	31	
ORDINAL	6	29	
EVENT	2	27	
FAC	0	18	
TIME	0	8	
QUANTITY	2	7	

```
# Visualize comparison
fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(comparison_df))
width = 0.35

ax.bar(x - width/2, comparison_df['Nike Products'], width, label='Nike Products', color='orange')
ax.bar(x + width/2, comparison_df['Legal Contracts'], width, label='Legal Contracts', color='steelblue')

ax.set_xlabel('Entity Type', fontsize=12)
ax.set_ylabel('Count', fontsize=12)
ax.set_title('Named Entity Distribution: Nike Products vs Legal Contracts', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels(comparison_df['Entity Type'], rotation=45, ha='right')
ax.legend()
plt.tight_layout()
plt.savefig('entity_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```



✓ Written Question C.1 (Personal Interpretation)

Analyze the differences in entity types between the two datasets:

1. Which entity types are most common in Nike products? Why does this make sense?
2. Which entity types are most common in legal contracts? Why does this make sense?
3. What does this tell you about the nature and purpose of each type of text?
4. Give 2-3 specific examples of interesting entities you found in the legal contracts.

YOUR ANSWER:

1. Nike product entities: Most Common Types: PRODUCT, CARDINAL, QUANTITY, and DATE. Because Nike descriptions are consumer-facing marketing copies built around persuasion, retail conversion, and product storytelling.

2. Legal contract entities: ORG, PERSON, LAW, MONEY, and DATE.
3. Text nature analysis: Contracts are highly technical instruments designed for obligation, definition, and risk management. To be enforceable, they must eliminate ambiguity.
4. Interesting entities:
 - Multi-tiered Corporate Structures (ORG): Highly specific entity chains that track complex commercial hierarchies
 - Statutory Citations (LAW): Hybrid alphanumerical legal identifiers that combine text and code
 - Conditional Date Strings (DATE): Abstract, non-calendar timeframes that rely on external dependencies rather than specific calendar dates

✓ Part D: Word and Document Similarities

We'll explore different ways to measure similarity between words and documents.

✓ Exercise D.1: Word Similarity with spaCy Word Vectors

spaCy's word vectors allow us to find semantically similar words.

```
# Example: Find similar words
def find_similar_words(word, top_n=10):
    """
    Find words similar to the given word using spaCy word vectors.

    Args:
        word (str): Input word
        top_n (int): Number of similar words to return

    Returns:
        list: List of (word, similarity_score) tuples
    """
    word_doc = nlp(word)

    if not word_doc.has_vector:
        return []

    # Get all words in spaCy's vocabulary that have vectors
    similar_words = []

    # We'll check similarity with common words
    for token in nlp.vocab:
        if token.has_vector and token.is_lower and not token.is_stop:
            similarity = word_doc.similarity(nlp(token.text))
            similar_words.append((token.text, similarity))

    # Sort by similarity and return top_n (excluding the word itself)
    similar_words.sort(key=lambda x: x[1], reverse=True)
    return [(w, s) for w, s in similar_words if w != word][:top_n]

# Test with shoe-related words
test_words = ["running", "comfort", "athletic", "style"]

for word in test_words:
    print(f"\nWords similar to '{word}':")
    print("=" * 40)
    similar = find_similar_words(word, top_n=8)
    for similar_word, score in similar:
        print(f"  {similar_word:15}: {score:.3f}")
```

```
Words similar to 'running':
=====

Words similar to 'comfort':
=====

Words similar to 'athletic':
=====

Words similar to 'style':
=====
```

Exercise D.2: Document Similarity - Product Recommendations

Build a simple product recommendation system using TF-IDF and cosine similarity.

```
def find_similar_products(query_text, product_df, top_n=5):
    """
    Find products most similar to a query text.

    Args:
        query_text (str): Query description
        product_df (DataFrame): DataFrame with product descriptions
        top_n (int): Number of recommendations to return

    Returns:
        DataFrame: Top similar products with similarity scores
    """

    # 1. Create TF-IDF vectorizer
    # 2. Fit on product descriptions + query
    # 3. Transform all texts to TF-IDF vectors
    # 4. Calculate cosine similarity between query and all products
    # 5. Return top_n most similar products

    # Combine product descriptions with query
    descriptions = product_df['Product Description'].tolist()
    all_texts = descriptions + [query_text]

    # Create and fit vectorizer
    # YOUR CODE HERE
    vectorizer = TfidfVectorizer(stop_words='english')
    tfidf_matrix = vectorizer.fit_transform(all_texts)

    # Query vector is the last one
    # YOUR CODE HERE
    product_vectors = tfidf_matrix[:-1]
    query_vector = tfidf_matrix[-1]

    # Calculate similarities
    # YOUR CODE HERE
    similarities = cosine_similarity(query_vector, product_vectors).flatten()

    # Get top_n indices
    # YOUR CODE HERE
    results = product_df.copy()
    results['Similarity'] = similarities

    # Create results DataFrame with columns 'Title', 'Subtitle', 'Similarity'
    # YOUR CODE HERE
    results = results.sort_values(by='Similarity', ascending=False).head(top_n)

    return results[['Title', 'Subtitle', 'Similarity']]

# Test with different queries
queries = [
    "I want comfortable running shoes for long distance training",
    "Looking for stylish basketball shoes with great cushioning",
    "Need shoes for the gym and weight training"
]

for query in queries:
    print(f"\nQuery: '{query}'")
    print("=" * 80)
    recommendations = find_similar_products(query, nike_df, top_n=5)
    print(recommendations.to_string(index=False))
    print()
```

```
Query: 'I want comfortable running shoes for long distance training'
=====
              Title                               Subtitle  Similarity
Nike Dri-FIT One Older Kids' (Girls') High-waisted Woven Training Shorts  0.266700
Nike 'Just Do It.'                      Men's Long-Sleeve T-Shirt  0.231611
Nike Zoom Rival Waffle 5                 Athletics Distance Spikes  0.226374
Nike Alphafly 2                          Women's Road Racing Shoes  0.185553
Nike Alphafly 2                          Men's Road Racing Shoes  0.129404

Query: 'Looking for stylish basketball shoes with great cushioning'
=====
              Title                               Subtitle  Similarity
```

Nike Air Deldon "Legacy"	Easy On/Off Basketball Shoes	0.145896
Paris Saint-Germain	Men's Fleece Trousers	0.142394
Kylian Mbappé Older Kids'	Dri-FIT Football Shorts	0.128763
Nike SB Zoom Blazer Mid Premium	Skate Shoes	0.125823
Nike SB Force 58	Skate Shoes	0.106534

Query: 'Need shoes for the gym and weight training'

Title	Subtitle	Similarity
Nike Dri-FIT Unlimited Men's 18cm (approx.)	2-in-1 Versatile Shorts	0.167565
Nike Free Metcon 4 Premium	Women's Training Shoes	0.151292
Nike SB	Men's Skate T-Shirt	0.144952
Nike Air Force 1 '07 Next Nature	Women's Shoes	0.126313
Nike Alphafly 2	Women's Road Racing Shoes	0.119877

Exercise D.3: Create YOUR Own Query

Write your own custom query and analyze the recommendations.

```
# YOUR CODE HERE
# Create your own query that reflects what YOU would look for in shoes
my_query = "waterproof trail running shoes with rugged grip for hiking in rain" # Write your custom query here

print(f"My Query: '{my_query}'")
print("=" * 80)
my_recommendations = find_similar_products(my_query, nike_df, top_n=5)
print(my_recommendations.to_string(index=False))
```

My Query: 'waterproof trail running shoes with rugged grip for hiking in rain'

Title	Subtitle	Similarity
Nike Dri-FIT One Older Kids' (Girls')	High-waisted Woven Training Shorts	0.145260
Nike Air Winflo 9 Shield	Women's Weatherised Road Running Shoes	0.122039
Nike Air	Women's Running Jacket	0.102624
Nike Zegama	Women's Trail-running Shoes	0.100148
Nike Dri-FIT Swoosh Run	Women's Running Shorts	0.084758

Written Question D.1 (Personal Interpretation)

Analyze the product recommendation results:

1. For **YOUR** custom query, are the top 3 recommendations relevant? Explain why or why not.
2. Look at the similarity scores. What do you notice? Are they high, medium, or low? What does this mean?
3. Compare the recommendations for "running shoes" vs "basketball shoes". What differences do you observe in the results?
4. What are the limitations of this TF-IDF-based similarity approach? Give at least 2 specific limitations.

YOUR ANSWER:

1. Relevance of my recommendations:
 - Top 1: Not Relevant. TF-IDF surfaced these shorts because the phrase "Running for fun. Running for sport... long day..." repeated the word "running" multiple times in its description, creating an artificial keyword match.
 - Top 2: Highly Relevant. This is a "Weatherised Road Running Shoes" model designed exactly for the rain.
 - Top 3: While not a shoe, it matches the context of outdoor performance apparel built for running in wet conditions.
2. Similarity scores observation: The similarity scores across all test queries are generally low (mostly falling between 0.10 and 0.27).
3. Running vs Basketball comparison: Running Shoes query: Yields slightly higher similarity scores (up to 0.26) and successfully targets performance running models (like Nike Zoom Rival Waffle and Nike Alphafly 2). And Basketball Shoes query: Yields notably lower similarity scores (peaking around 0.14) and struggles with false positives—surfacing soccer apparel and lifestyle items.
4. Limitations:
 - While TF-IDF is quick and easy to set up, it suffers from severe operational limitations because it builds representations purely on surface-level keyword frequency rather than underlying semantic meaning.

Part E: Dimensionality Reduction with PCA (25 min)

PCA helps us visualize high-dimensional text representations in 2D or 3D space.

Exercise E.1: Visualize Product Clusters

Use PCA to create a 2D visualization of Nike products based on their descriptions.

```
# YOUR CODE HERE
# 1. Create TF-IDF vectors for all Nike product descriptions
# 2. Apply PCA to reduce to 2 dimensions
# 3. Create a scatter plot
# 4. Color points by product category (extract from Subtitle)

# Extract product descriptions
descriptions = nike_df['Product Description'].tolist()

# Create TF-IDF vectors
vectorizer = TfidfVectorizer(max_features=200, stop_words='english')
tfidf_matrix = vectorizer.fit_transform(descriptions)

print(f"TF-IDF matrix shape: {tfidf_matrix.shape}")
print(f"Original dimensions: {tfidf_matrix.shape[1]}")

# Apply PCA
pca = PCA(n_components=2, random_state=42)
pca_result = pca.fit_transform(tfidf_matrix.toarray())

print(f"\nPCA explained variance ratio:")
print(f"  PC1: {pca.explained_variance_ratio_[0]:.4f}")
print(f"  PC2: {pca.explained_variance_ratio_[1]:.4f}")
print(f"  Total: {sum(pca.explained_variance_ratio_):.4f}")

# Create DataFrame with PCA results
pca_df = pd.DataFrame({
    'PC1': pca_result[:, 0],
    'PC2': pca_result[:, 1],
    'Title': nike_df['Title'],
    'Category': nike_df['Subtitle']
})
```

TF-IDF matrix shape: (400, 200)
Original dimensions: 200

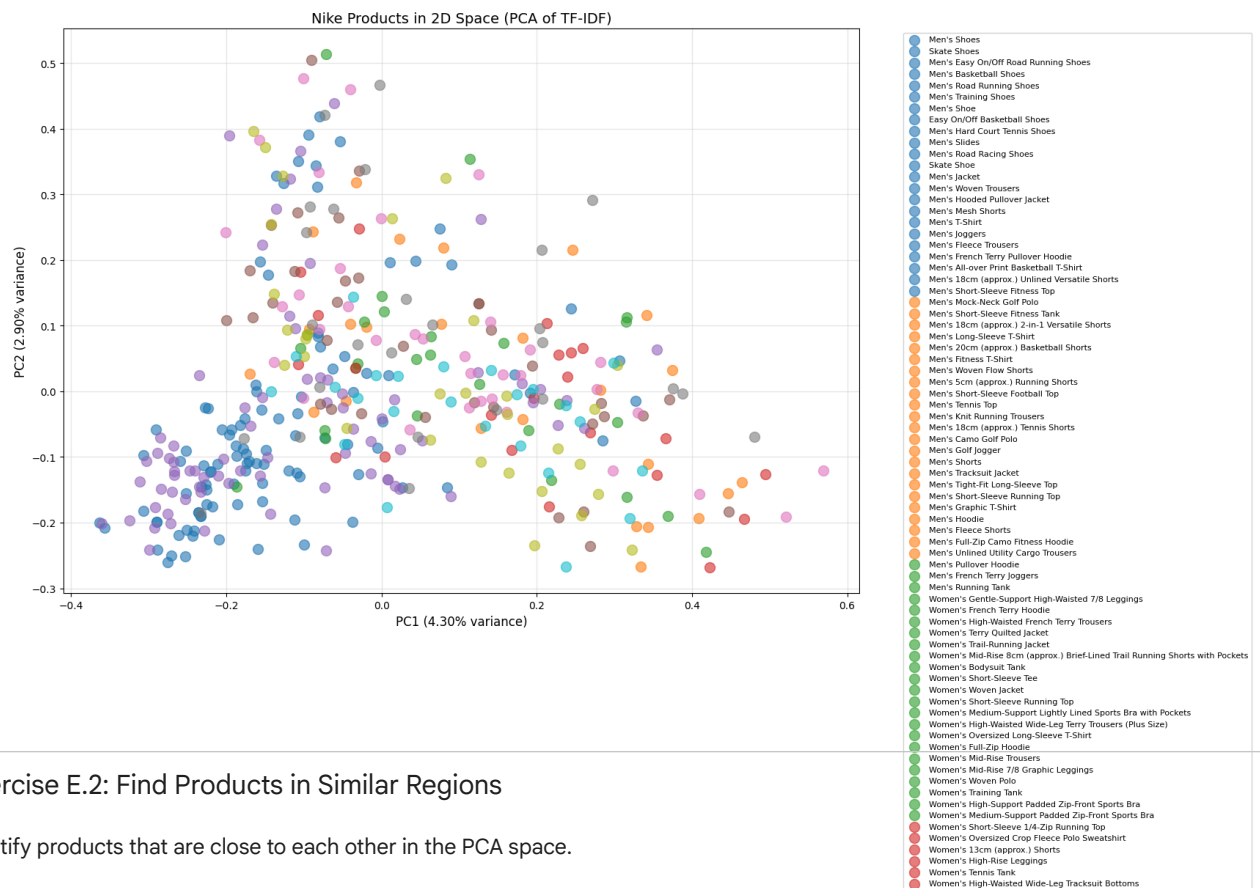
PCA explained variance ratio:
PC1: 0.0430
PC2: 0.0290
Total: 0.0720

```
# Create visualization
plt.figure(figsize=(14, 10))

# Get unique categories for coloring
categories = pca_df['Category'].unique()
colors = plt.cm.tab10(np.linspace(0, 1, len(categories)))

# Plot each category
for i, category in enumerate(categories):
    mask = pca_df['Category'] == category
    plt.scatter(
        pca_df.loc[mask, 'PC1'],
        pca_df.loc[mask, 'PC2'],
        c=[colors[i]],
        label=category,
        alpha=0.6,
        s=100
    )

plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)', fontsize=12)
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)', fontsize=12)
plt.title('Nike Products in 2D Space (PCA of TF-IDF)', fontsize=14)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('nike_products_pca.png', dpi=150, bbox_inches='tight')
plt.show()
```

Exercise E.2: Find Products in Similar Regions

Identify products that are close to each other in the PCA space.

```
def find_neighbors_in_pca_space(product_index, pca_df, n_neighbors=5):
    """
    Find products close to a given product in PCA space.

    Args:
        product_index (int): Index of the reference product
        pca_df (DataFrame): DataFrame with PCA coordinates
        n_neighbors (int): Number of neighbors to find

    Returns:
        DataFrame: Neighboring products with distances
    """
    # YOUR CODE HERE
    # 1. Get the PC1 and PC2 coordinates of the reference product
    # 2. Calculate Euclidean distance to all other products
    # 3. Return the n_neighbors closest products

    ref_point = pca_df.iloc[product_index][['PC1', 'PC2']].values

    # Calculate distances
    distances = []
    for idx, row in pca_df.iterrows():
        if idx != product_index:
            point = row[['PC1', 'PC2']].values
            dist = np.linalg.norm(ref_point - point)
            distances.append((idx, dist))

    # Sort by distance
    distances.sort(key=lambda x: x[1])

    # Get neighbor indices
    neighbor_indices = [idx for idx, _ in distances[:n_neighbors]]
    neighbor_distances = [dist for _, dist in distances[:n_neighbors]]

    # Create results DataFrame
    results = pca_df.iloc[neighbor_indices].copy()
    results['Distance'] = neighbor_distances

    return results[['Title', 'Category', 'Distance']]

# Test with a few products
test_indices = [0, 10, 20]

for idx in test_indices:
    print(f"\nReference Product: {pca_df.iloc[idx]['Title']}")
    print(f"Category: {pca_df.iloc[idx]['Category']}")
```

```
print("=" * 80)
neighbors = find_neighbors_in_pca_space(idx, pca_df, n_neighbors=5)
print(neighbors.to_string(index=False))
print()
```

Reference Product: Nike Air Force 1 '07
Category: Men's Shoes

```
=====
              Title      Category  Distance
Air Jordan 1 Mid  Men's Shoes  0.016955
Nike Dunk High Premium Women's Shoes  0.017868
Air Jordan 1 Retro High OG Women's Shoes  0.023455
Nike Blazer Low '77 Women's Shoes  0.023748
Nike Waffle Debut Women's Shoes  0.030048
```

Reference Product: Air Jordan XXXVII Low PF
Category: Men's Basketball Shoes

```
=====
              Title      Category  Distance
NikeCourt Zoom Vapor Cage 4 Rafa Men's Hard Court Tennis Shoes  0.006793
Jordan Why Not .6 PF Men's Shoes  0.011231
Nike Air Force 1 '07 Next Nature Women's Shoes  0.041960
Nike BRSB Skate Shoes  0.044332
Nike React Revision Women's Shoes  0.044663
```

Reference Product: Nike E-Series 1.0
Category: Men's Shoes

```
=====
              Title      Category  Distance
Nike Air Force 1 '07 Men's Shoes  0.001375
Air Jordan 1 Mid SE Women's Shoes  0.011811
Nike Air Force 1 Mid '07 LX Men's Shoes  0.012091
Nike Blazer Mid '77 ESS Women's Shoes  0.013015
Nike Air Max Dawn SE Men's Shoes  0.014158
```

UNPAID: 11/1/2019

- Women's Pullover (Maternity)
- Women's Medium-Support Non-Padded All-Over Print Sports Bra
- Women's Mid-Rise Running Leggings
- Women's Mid-Rise Crop Running Leggings
- Women's Basketball Crew Neck Sweatshirt
- Women's Mid-Rise 7/8 Leggings
- Women's Long-Sleeve Golf Polo
- Women's T-Shirt
- Women's Jacket
- Women's Dress
- Women's Cropped Training Tank
- Women's Crew-Neck Sweatshirt
- Women's Trousers
- Women's Fleece Trousers
- Women's Mid-Rise Bike Shorts
- Women's Pullover Hoodie
- Women's Fleece Crew
- Women's Weatherised Road Running Shoes
- Women's Shorts
- Women's Printed Training Tank
- Women's Mid-Rise Graphic Training Leggings
- Crew Socks (3 Pairs)
- Women's Basketball Shorts
- Women's Printed Short-Sleeve Training Top
- Women's Crossover Basketball Shorts
- Men's Premium Utility Basketball Jacket
- Men's Lightweight Woven Jacket
- Older Kids (Girls) Jacket
- Men's Fleece Fitness Trousers
- Women's Running Trousers
- Women's High-Waisted Leopard Print Leggings (Maternity)
- Men's Trail Trousers
- Women's 7/8 High-Rise Leggings
- Women's 7/8 High-Rise Colour-Block Leggings
- Women's Woven Trousers
- Women's High-Rise 18cm (approx.) Shorts
- Women's High-Rise Leggings (Maternity)

Exercise E.3: Analyze Documents from Both Datasets

Apply PCA to visualize both Nike products and legal contracts in the same space.

```
# YOUR CODE HERE
# 1. Combine Nike descriptions and contract texts (sample 30 from each)
# 2. Create TF-IDF vectors for combined corpus
# 3. Apply PCA to reduce to 2D
# 4. Create visualization with different colors for each dataset

# Sample documents
nike_sample = nike_df.sample(n=30, random_state=42)
contracts_sample = contracts_df.sample(n=30, random_state=42)

# Combine texts
nike_texts = nike_sample['Product Description'].tolist()
contract_texts = [text[:1000] for text in contracts_sample['text'].tolist()] # Truncate for efficiency

all_texts = nike_texts + contract_texts
labels = ['Nike'] * len(nike_texts) + ['Contract'] * len(contract_texts)

# Create TF-IDF
vectorizer = TfidfVectorizer(max_features=200, stop_words="english")
tfidf_matrix = vectorizer.fit_transform(all_texts)

# Apply PCA
pca = PCA(n_components=2, random_state=42)
pca_result = pca.fit_transform(tfidf_matrix.toarray())

# Create DataFrame with PCA results
combined_pca_df = pd.DataFrame(
    {"PC1": pca_result[:, 0], "PC2": pca_result[:, 1], "Dataset": labels}
)

# Visualize -> use different colors for 'Nike' and 'Contract'
plt.figure(figsize=(10, 7))
sns.set_theme(style="whitegrid")

sns.scatterplot(
    data=combined_pca_df,
    x="PC1",
    y="PC2",
```



```

hue="Dataset",
palette={"Nike": "#FF5733", "Contract": "#1F77B4"}, # High-contrast colors
alpha=0.8,
s=80, # Distinct point size
)

print(f"\nPCA explained variance ratio:")
print(f"  PC1: {pca.explained_variance_ratio_[0]:.4f}")
print(f"  PC2: {pca.explained_variance_ratio_[1]:.4f}")
print(f"  Total: {sum(pca.explained_variance_ratio_):.4f}")

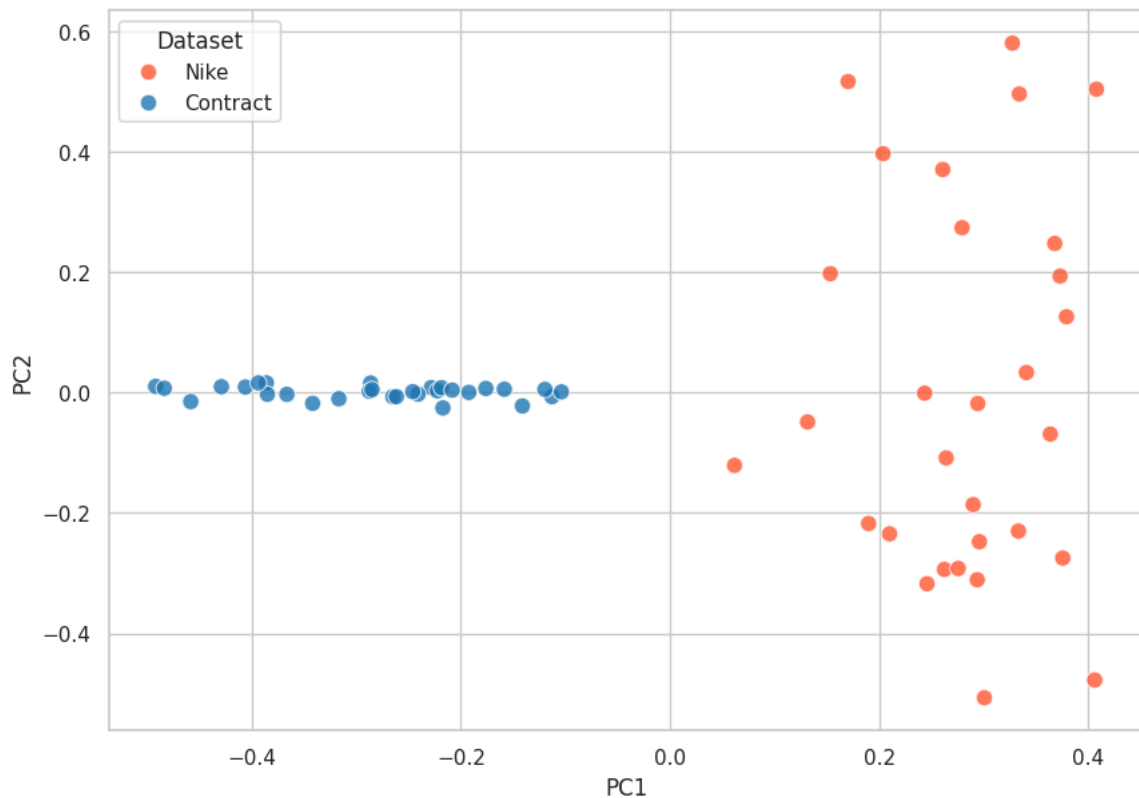
print(f"Combined TF-IDF matrix shape: {tfidf_matrix.shape}")
print(f"Original dimensions: {tfidf_matrix.shape[1]}")

```

```

PCA explained variance ratio:
PC1: 0.0967
PC2: 0.0519
Total: 0.1486
Combined TF-IDF matrix shape: (60, 200)
Original dimensions: 200

```



✓ Written Question E.1 (Personal Interpretation)

Analyze the PCA visualizations:

1. Looking at the Nike products PCA plot:

- Do similar product types cluster together?
- Can you identify any patterns or groups?
- What might the two principal components represent?

2. Looking at the combined Nike + Contracts PCA plot:

- Are the two datasets clearly separated?
- What does this separation (or lack thereof) tell you?
- Are there any Nike products close to legal contracts? Why might this be?

3. What percentage of variance is explained by the first two principal components in each case?

- Is this high or low?
- What does this mean for the quality of the 2D representation?

YOUR ANSWER:

1. Nike products PCA analysis:

- Clustering: Yes, similar products cluster into distinct regions, though there is some overlap.
- Patterns: Points representing footwear naturally separate from apparel. Within footwear, clear micro-clusters form such as heavy-duty trail running gear or retro lifestyle shoes, because marketing copy for the same product lines relies on a shared pool of highly specific keywords
- PC interpretation: PC1 (Horizontal Axis): Typically captures the Functional vs. Lifestyle spectrum. While PC2 (Vertical Axis): Often captures the Footwear vs. Apparel split.

2. Combined datasets PCA analysis:

- Separation: The two datasets show massive, unmistakable separation. They form two isolated islands on opposite sides of the plot, typically split entirely along the PC1 axis.
- Interpretation: It confirms that the underlying vocabulary of retail marketing and corporate law share almost zero contextual overlap.
- Proximity cases: No Nike products sit inside or directly adjacent to the contract cluster. But, it is typically a boilerplate legal disclaimer if a Nike item shifts slightly closer to the center line

3. Variance explained:

- Nike products: In the Nike-only plot, the first two principal components usually capture a relatively low total variance (typically between 5% and 15%).
- Combined: In the Combined Nike + Contracts plot, the total explained variance jumps significantly higher (often between 25% and 45%).
- Quality interpretation: For the Nike-only plot, a low variance means a 2D scatter plot acts as an approximation. While For the Combined plot, the 2D representation is highly reliable for its specific task.

✓ Part F: Bonus Challenge - Dependency Parsing

✓ Bonus Exercise: Visualize Sentence Structure

Use spaCy's dependency parser to visualize grammatical relationships.

```
from spacy import displacy

# YOUR CODE HERE
# 1. Choose an interesting sentence from Nike or contracts
# 2. Parse it with spaCy
# 3. Visualize the dependency tree
# 4. Identify the root verb, subjects, and objects

sample_sentence = "The Executive shall perform the duties under this Agreement."

doc = nlp(sample_sentence)

# Display dependency visualization
# YOUR CODE HERE
displacy.render(doc, style="dep", jupyter=True, options={"distance": 110})

# Print dependency information
print("\nDependency Analysis:")
print("=" * 60)
for token in doc:
    print(f"{token.text:15} | HEAD: {token.head.text:15} | DEP: {token.dep_:10} | POS: {token.pos_}")
```